SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Experiment

COURSE NAME: Data Structures **COURSE CODE: CSA0310**

COURSE OUTCOME COVERED

CO: *Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees (BL3, BL5)*

Assessment Tool Weightage: 40%

Questions: EACH QUESTIONS CARRIES : 10 MARKS Duration :ONE HOUR

1. Write a C program to develop a Binary Search Tree (BST) using the dataset 45, 20, 60, 10, 30, 50, 70, 25, 35 and perform the required operations, and document the results.
    1. Insert all elements
    2. Perform inorder, preorder, postorder traversal
    3. Delete node 20
    4. Find height of tree
    5. Search for key 25
2. Experiment to construct an AVL Tree using the elements 50, 30, 70, 20, 40, 60, 80, perform the required operations using a C program, and document the results.
    1. Delete node 30
    2. Rebalance the tree
    3. Show all rotations involved
    4. Compare height before and after deletion
3. Understanding the implementation of a B-Tree of order *m* with insertion operations using the elements 35, 45, 12, 67, 10, and 12 through a C program, and documenting the results.
4. Write a C program to implement a Splay Tree with Zig, Zig-Zig, and Zig-Zag operations for the data set 10, 45, 8, 25, 75, and document the results.
5. Experiment to implement a Red-Black Tree with insertion, deletion, and traversal operations using a C program, and document the results.
6. Write a C program to construct a Binary Search Tree using given inputs. Implement insertion and traversal operations. Construct the tree and display inorder, preorder, and postorder results.
7. Write a program to construct a BST from a set of elements and delete a node with different cases and Construct the updated tree and display the result.

8. Write a program to construct a Binary Search Tree. Implement functions to calculate height and count total nodes and Construct the tree and display computed values with explanation.

9. Write a program that:
Inserts N = 1000 keys in increasing order into a Splay Tree (worst case)
Performs M = 5000 searches with a Zipfian distribution (some keys accessed much more frequently)
   - Tracks:
   1. Total time for all operations
   2. Individual operation times
   3. Height before and after each operation
   4. Number of rotations per operation

10. Construct a BST from a given sequence of integers: [50,40,60,70,80], then write a function to delete the root node and restructure the tree.

11. Implement a full AVL tree with detailed balance tracking Implement an AVL tree with:
    1. Insertion with all four rotations (LL, RR, LR, RL)
    2. Deletion with rebalancing (all cases)
    3. Function to display tree with balance factors
    4. Function to verify AVL property (for every node, $|bf| \leq 1$)
    5. Function to compute total rotations performed so far
    6. Function to return all nodes that are unbalanced after a given insertion (before fixing)

12. Implement a B-tree of order 5 with disk block simulation
Implement a B-tree where each node can have max 4 keys (order 5, min degree t=3). Include:
    1. Insertion with split logic (split middle key up)
    2. Deletion with:
    3. Borrow from left sibling
    4. Borrow from right sibling
    5. Merge with siblings
    6. Search with path logging
    7. Function to print tree structure with node levels

13. Construct B-trees of orders 3, 4, and 5 (minimum degree t = 2, 3, 4) by inserting: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150]
For each tree:
    1. Draw the final tree structure (or print level by level)
    2. Count total number of nodes
    3. Count total number of splits performed
    4. Compute average node occupancy percentage
    5. Write a function that finds the minimum order required so that tree height ≤ 2 for N=1000 keys.

14. Write a program to construct a Red-Black Tree and perform deletion operation finally
Construct the updated tree and display structure.

15. Implement a menu-driven program that allows inserting into BST, AVL, or Red-Black tree based on user choice.

16. Implement a function that converts a given 2-3-4 tree (B-tree of order 4) into a Red-Black tree:
    1. 2-node → black node
    2. 3-node → black node with one red child
    3. 4-node → black node with two red children
    4. Construct a 2-3-4 tree by inserting:
       [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
    5. Then convert it to Red-Black tree.
    6. Write a function to convert the Red-Black tree back to 2-3-4 tree and verify it matches the original.

17. Write a program to construct a Splay Tree and implement zig, zig-zig, and zig-zag rotations. Then display each step of transformation.

18. Write a C program to construct a Binary Search Tree (BST) using a set of input values and perform insertion and all three traversal techniques (inorder, preorder, postorder).
    and display the traversal outputs clearly.

19. Design a hybrid tree that starts as a plain BST. After every k insertions, check if the tree height > 2*log2(N). If yes, rebuild the entire tree as a balanced AVL tree. This is similar to "scapegoat tree" idea.
    Implement:
    1. Normal BST insertion (without rotations)
    2. A function is_balanced() that checks height condition
    3. A function rebuild_to_avl() that extracts all keys, sorts them, and builds an AVL tree
    4. A counter for number of rebuilds

20. Simulate a disk with 4KB blocks. Each node in B-tree = 1 block. Each node in Red-Black tree = 1 block (but RB tree has many small nodes).
    Parameters:
    1. B-tree order: max 100 keys per node (to fit in 4KB)
    2. Red-Black tree: each node occupies 1 block (inefficient)

    Construct both trees by inserting 10,000 random integers. Simulate disk I/O: every node access = 1 I/O operation.

21. Construct a Trie data structure for a given set of strings, implement insertion, searching, and deletion operations, and analyze its efficiency in terms of time complexity for prefix-based searches.

22. Design and implement a 2-3 Tree for a sequence of integer inputs, demonstrating how node splitting and tree balancing occur during insertion and deletion operations.

23. Develop a 2-3-4 Tree using a suitable programming language, insert a set of keys, and illustrate how the tree maintains balance through splitting and merging of nodes.

24. Implement a Red-Black Tree and demonstrate all balancing cases during insertion, including recoloring and rotations, with step-by-step explanation.

25. Implement a Quad Tree for image representation, showing how it compresses uniform regions and improves storage efficiency.

26. Design and implement an Octree for 3D spatial data, demonstrating subdivision of space and its application in 3D graphics or collision detection.

27. Compare the performance of Trie, Red-Black Tree, and Splay Tree by implementing them for a common dataset and analyzing their search and insertion efficiencies.

28. Construct a Quad Tree for geographic data representation and perform range queries to retrieve all points within a specified region.

29. Construct a Quad Tree for geographic data representation and perform range queries to retrieve all points within a specified region.

30. Construct and analyze different tree structures (2-3 Tree, 2-3-4 Tree, Red-Black Tree, and Splay Tree) for the same dataset and evaluate their height, balance, and performance.

31. Construct a Trie using the words *{cat, car, care, dog, dot}*, perform insertion for all words, search for the prefix "ca", and delete the word "car" while maintaining the Trie structure.

32. Build a 2-3 Tree by inserting the elements *{15, 25, 5, 10, 20, 30}*, and demonstrate node splitting during insertion along with an inorder traversal of the final tree.

33. Create a 2-3-4 Tree for the dataset *{50, 40, 60, 30, 70, 20, 80}*, perform insertions step-by-step, and show how 4-nodes are split during the process.

34. Implement a Red-Black Tree for the keys *{10, 20, 30, 15, 25}*, and illustrate all rotations and recoloring steps required to maintain Red-Black properties after each insertion.

35. Construct a Red-Black Tree using *{45, 35, 55, 25, 40, 50, 60}*, delete the node *35*, and explain how violations are fixed using rotations and recoloring.

36. Develop a Splay Tree with elements *{100, 50, 200, 40, 30, 20}*, perform a search operation for *20*, and demonstrate the zig-zig rotations that bring it to the root.

37. Implement a Splay Tree using the sequence *{10, 20, 30, 40, 50}*, perform repeated searches for *30*, and analyze how the structure adapts to frequent access.

38. Construct a Quad Tree for a 2D space of size 8×8 with points *(1,2), (3,5), (6,6), (7,1)*, and perform a range query to retrieve points within the region *(2,2) to (6,6)*.

39. Design a Quad Tree for a binary image matrix, perform recursive decomposition, and identify homogeneous regions to demonstrate compression.

40. Build an Octree for 3D points *{(1,1,1), (2,3,4), (5,5,5), (7,8,6)}*, perform insertion, and execute a nearest neighbor search for the point *(2,2,2)*.

Code of Conduct:

I GVENKATA GANESH 192425381*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G V GANESH

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

CO3 AT2 DATA STRUCTURES

Q.1] Write a program to construct a BST from a set of elements and delete a node with different cases and Construct the updated tree and display the result.

Ans:

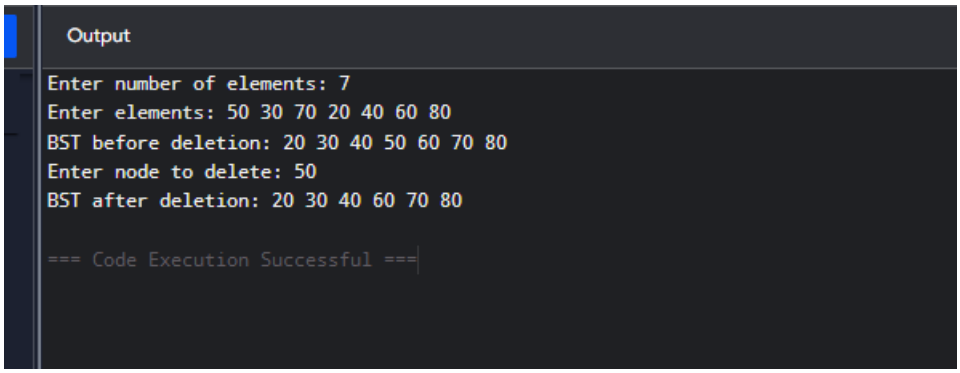
PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *left, *right;
};
struct node* create(int x) {
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = x;
    newnode->left = newnode->right = NULL;
    return newnode;
}
struct node* insert(struct node* root, int x) {
    if (root == NULL)
        return create(x);
    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);
    return root;
}
struct node* findMin(struct node* root) {
    while (root->left != NULL)
```

```
        root = root->left;
    return root;
}
struct node* deleteNode(struct node* root, int x) {
    struct node* temp;
    if (root == NULL)
        return root;
    if (x < root->data)
        root->left = deleteNode(root->left, x);
    else if (x > root->data)
        root->right = deleteNode(root->right, x);
    else {
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        if (root->left == NULL) {
            temp = root->right;
            free(root);
            return temp;
        }
        if (root->right == NULL) {
            temp = root->left;
            free(root);
            return temp;
        }
        temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
```

```
    }
    return root;
}
void inorder(struct node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
int main() {
    struct node* root = NULL;
    int n, x, del, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }
    printf("BST before deletion: ");
    inorder(root);
    printf("\nEnter node to delete: ");
    scanf("%d", &del);
    root = deleteNode(root, del);
    printf("BST after deletion: ");
    inorder(root);
    return 0;
}
```

OUTPUT


Output
Enter number of elements: 7
Enter elements: 50 30 70 20 40 60 80
BST before deletion: 20 30 40 50 60 70 80
Enter node to delete: 50
BST after deletion: 20 30 40 60 70 80
=== Code Execution Successful ===